

Depth-First Search Continued: Topological Sort and Strongly-Connected Components

Design and Analysis of Algorithms

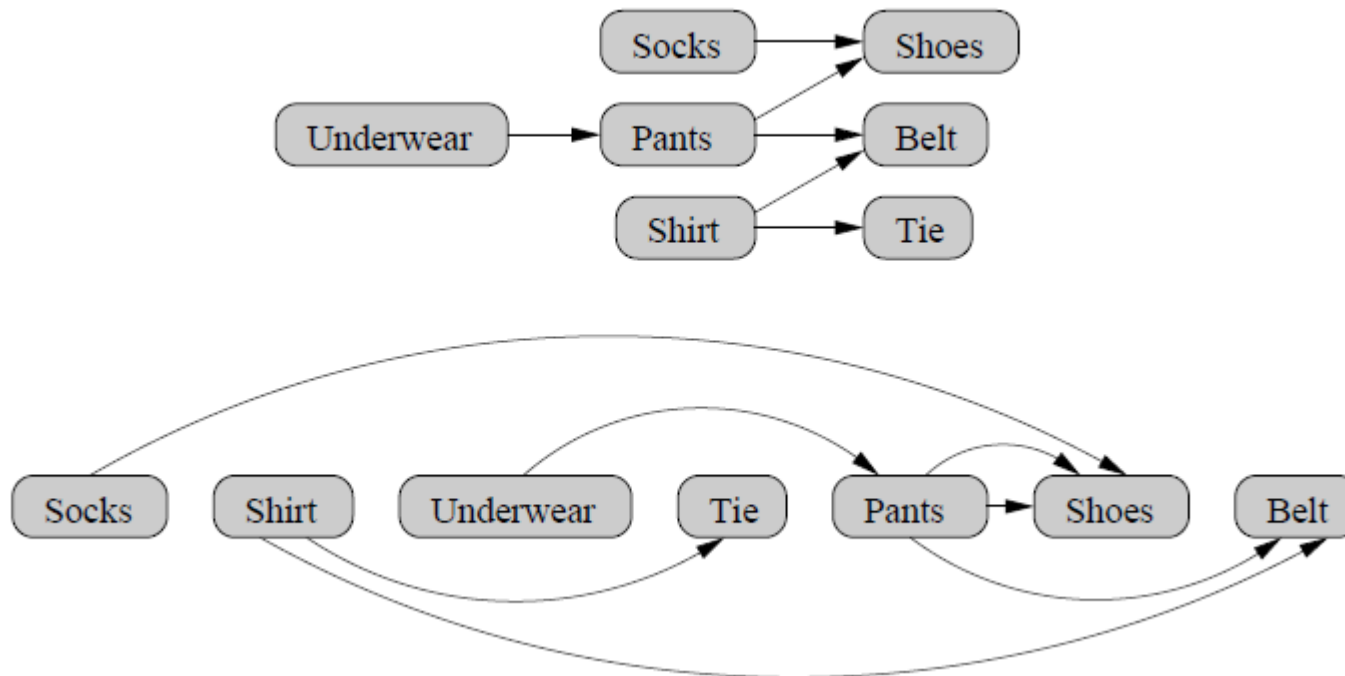
Brian C. Dean

**School of Computing
Clemson University**



Topological Sorting

- Directed Acyclic Graphs (DAGs) are often used to model systems with “precedence” constraints.
- Topological sorting is the process of ordering the nodes of a DAG so all edges point a consistent direction.



Topological Sorting

- There are several ways to topologically sort in $O(m + n)$ time; for example:
 - Find a node with no incoming edges, add it next to the ordering, remove it from our graph, repeat.
(why must such a node always exist?)
 - If we ever find that every node has an incoming edge, then our graph must contain a cycle (so this gives an alternate way to do cycle detection).
- Depth-first search gives us another very simple topological sorting algorithm...

Topological Sorting

- Let's associate with each node i a “discovery time” $d[i]$ and a “finishing time” $f[i]$:

```
DFS-Visit( $i$ ):  
  Status[ $i$ ] = 'visited'  
   $d[i]$  = current_time, Increment current_time  
  For all  $j$  such that  $(i, j)$  is an edge:  
    If Status[ $j$ ] = 'unvisited': DFS-Visit( $j$ )  
   $f[i]$  = current_time, Increment current_time
```

- To topologically sort a DAG, just perform a Full-DFS and then output nodes in reverse order of finishing times.
 - We can output each node when it is “finished”, or we can sort the nodes by their finishing times (in linear time with counting sort).

Topological Sorting

- **Claim:** In a DAG with an edge from node i to node j , we will have $f(i) > f(j)$.
- **Proof:** Consider two cases:
 - (a) Full-DFS visits i first.
 - (b) Full-DFS visits j first.

In both cases, we have $f(i) > f(j)$ as long as our graph contains no directed cycles (which it doesn't, since it's a DAG!)

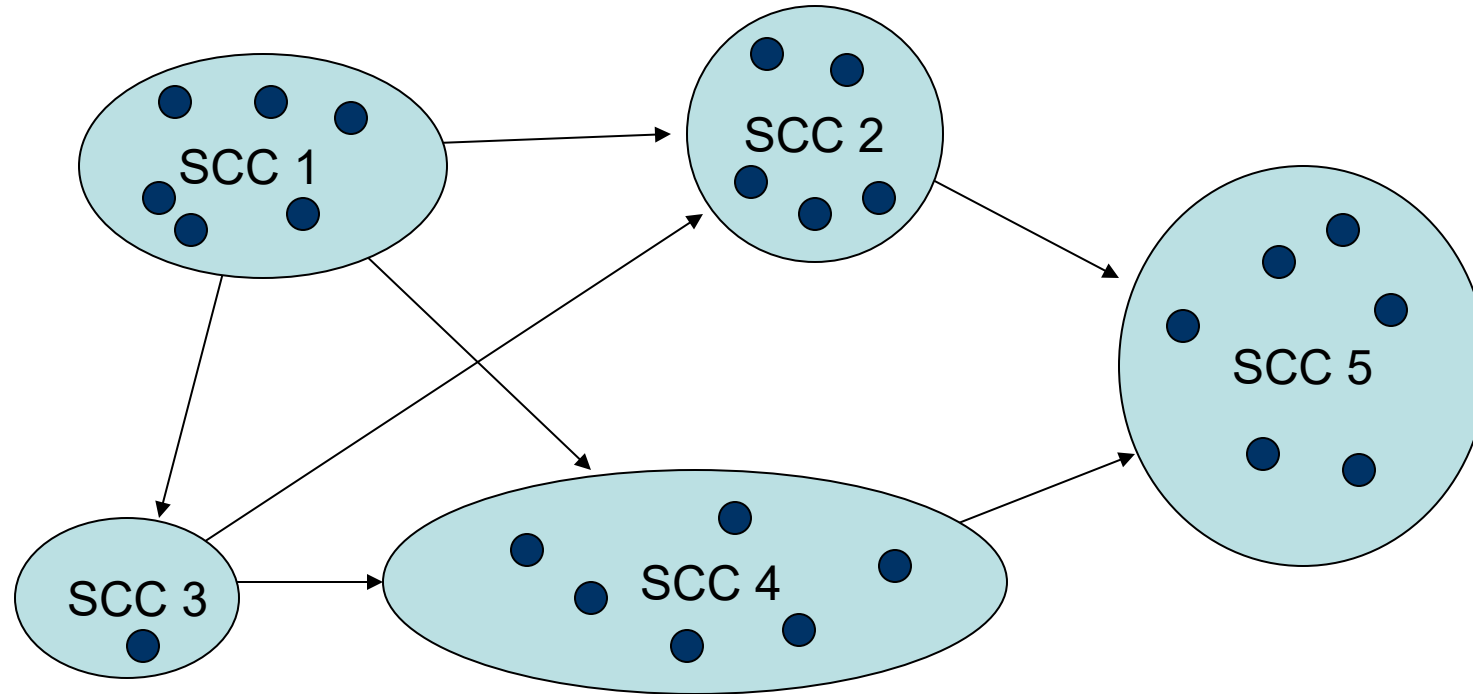
Strongly-Connected Components

- We have already seen how DFS partition an undirected graph into connected components.
- Although it's slightly less obvious, we can also use DFS to partition a directed graph into strongly connected components!
 - Recall that two nodes i and j belong to the same strongly connected component if there is a directed path from both i to j and from j to i .

Strongly-Connected Components

- Consider this algorithm:
 - Full-DFS
 - Reverse all the edges in our graph
 - Full-DFS again, processing nodes in decreasing order of finishing time (according to the 1st DFS).
- **Claim:** In the second Full-DFS, each call to DFS-Visit will end up visiting precisely the set of nodes in one of the strongly connected components in our graph!
- So we can partition a graph into its strongly connected components in $O(m + n)$ time.

Strongly-Connected Components: Topological Sorting in Disguise!



Claim: For any two SCCs A and B with edges from A to B,
$$\max_{i \in A} f[i] > \max_{j \in B} f[j]$$